TECHADEMY

FORGE — FDE ACADEMY · RAG FOUNDATIONS ON AZURE · Day 6 . 30-Jun-2026

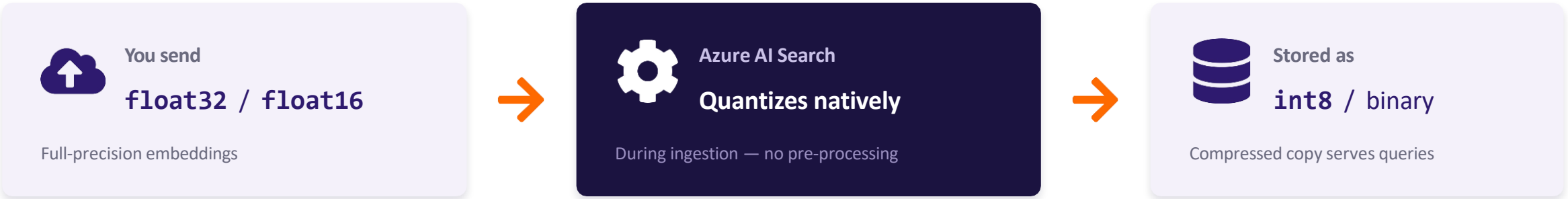
Vector Quantization in Azure AI Search

Configuring built-in vector compression to shrink the index — before you ever push a vector.

THE CORE IDEA

Quantization is a property of the index — not your code

You configure built-in vector compression inside the index definition. You send standard full-precision vectors; Azure AI Search compresses them automatically before they hit disk and memory.



Connective spine: *retrieval quality is RAG quality.* Quantization keeps a large vector index affordable and fast at scale — without surrendering recall, because rescoring closes most of the quality gap.

The originals (when retained) are used only for a precise second-pass re-ranking of the top candidates.

Two compression modes, big footprint cuts

Both modes reduce memory and disk for float16 / float32 embeddings. Choose by how aggressively you need to cut storage versus how much quality headroom you have.

≈4×

Scalar Quantization

float32 ⟶ int8
Low, predictable quality loss

≈32×

Binary Quantization

each dimension ⟶ 1 bit
Max compression for huge indexes

≤92.5%

Storage cost cut

binary + discardOriginals
relative to no compression

Rescoring closes the gap. In Microsoft's own tests, quality stayed at or near baseline when originals were preserved and rescoring was enabled — measured by nDCG@k.

THE WORKFLOW

Five steps to a quantized index

All configuration lives in the **`vectorSearch`** block of your index schema.

1

Add a compression config

Define SQ or BQ under vectorSearch.compressions.

2

Configure rescoring

Enable rescoring + oversampling to protect recall.

3

Map to a vector profile

Tie an algorithm (HNSW) to the compression.

4

Apply profile to field

Field type stays Collection(Edm.Single).

5

Push data normally

Send raw embeddings; quantization is automatic.

Order of build: define → tune → bind → attach → ingest. Everything before “push data” is one-time schema setup.

Add a compression configuration

```
"compressions": [
  {
    "name": "my-scalar-compression",
    "kind": "scalarQuantization",
    "scalarQuantizationParameters": {
      "quantizedDataType": "int8"
    },
    "rescoringOptions": {
      "enableRescoring": true,
      "defaultOversampling": 2.0,
      "rescoreStorageMethod": "preserveOriginals"
    }
  }
]
```

Scalar (SQ)

Each float32 dimension ⟶ an int8 byte. Sensible default; modest quality loss.

Binary (BQ)

Each dimension ⟶ a single bit (0/1). Max compression for very large indexes.

API version note: older guidance shows a flat **`rerankWithOriginals`**. The current API replaces it with the nested **`rescoringOptions`** object — pin your api-version and verify against the live reference.

Configure rescoring & oversampling

Quantization is lossy. To protect recall, configure these inside rescoringOptions:

`enableRescoring` `true`

Runs a precise second pass on top candidates using full-precision vectors, then re-ranks.

`defaultOversampling` `2.0 – 3.0`

A multiplier on K. With K=10 and 2×, the engine pulls 20 candidates, then narrows to the best 10.

`rescoreStorageMethod` `preserve / discard`

SQ must preserveOriginals. BQ may discardOriginals and rescore via the dot product.

The recall lever

Oversampling trades a little latency for accuracy.

Start at 2.0

Raise only if evaluation (Recall@K, nDCG) shows the compressed index dropping relevant results.

You can also override oversampling per query — no index change needed.

Note: rescoring & oversampling apply to HNSW queries. With exhaustive KNN every vector is already in scope, so they have no effect.

STEPS 3 & 4

Bind a profile, then attach it to the field

3 Map compression to a vector profile

```
"profiles": [
  {
    "name": "my-quantized-profile",
    "algorithm": "my-hnsw-algorithm",
    "compression": "my-scalar-compression"
  }
]
```

Ties your search algorithm (**HNSW** / **exhaustiveKnn**) to the compression you defined.

4 Apply the profile to your vector field

```
"fields": [
  {
    "name": "description_vector",
    "type": "Collection(Edm.Single)",
    "dimensions": 1536,
    "searchable": true,
    "retrievable": true,
    "vectorSearchProfile":
        "my-quantized-profile"
  }
]
```

Field type stays full-precision — **Collection(Edm.Single)** or **Edm.Half**. Quantization happens behind the scenes.

Step 5 — Push data normally: no pre-quantizing in Python or C#. Send raw, full-precision embeddings via your indexer or push API; Azure quantizes natively during ingestion.

Scalar vs Binary — choosing a mode

Dimension	Scalar Quantization	Binary Quantization
Compression	≈4× (`float32` → `int8`)	Up to ≈32× (dimension → 1 bit)
Quality loss	Low and predictable	Higher; mitigated by rescoring
Originals	Required (`preserveOriginals`)	Optional (can `discardOriginals`)
Rescoring uses	Original full-precision vectors	Originals, or the binary dot product
Best for	Default for most enterprise RAG	Very large indexes; storage-bound

KEY TAKEAWAYS

Six things to remember

Declarative, not procedural

Quantization is a property of the index schema — not a step in your ingestion code.

Send full precision

The service compresses to `int8` or binary on the way to disk and memory.

Always rescore

Pair lossy compression with `enableRescoring` + a sensible `defaultOversampling`.

Mind the originals

Scalar must `preserveOriginals`; binary can `discardOriginals` for max savings.

HNSW gets the benefit

Rescoring & oversampling apply to `HNSW` queries, not exhaustive KNN.

Pin the API version

The `rescoringOptions` layout has changed; verify field names against the live reference.